2011 Room Reverb

How the software works

Architecture, room model, FDN engine, Play vs Render, mic array and colour, parameters, known issues and roadmap — with 25 figures

Code snapshot: 28 Sep 2026, about 09:05 BST (byte-identical to the 27 Sep 2026 snapshot)

Source folder: `Desktop\2011 Room Reverb\software` (read-only copy)

Mic colour stage: Currently FIR; migrating to a 3-biquad IIR cascade (in progress by Grok Build, started 09:33 BST, 28 Sep 2026 — after this snapshot)

Prepared: Monday 28 September 2026, for Idle Hanz

Tags used throughout: VERIFIED checked in code or by running it · ASSUMED inferred / design note / not listened to · PLANNED planned or in progress

Contents

1 What this document covers 3

2 Big picture: architecture 4

3 The browser app 5

3.1 Tabs and sidebar 5

3.2 How the UI talks to Python 6

4 The room model 9

4.1 Geometry and coordinates 9

4.2 Image sources and the 32 voices 9

5 The FDN engine 11

6 Per-tap processing: early, air and surfaces 12

6.1 Air 13

6.2 Surfaces 14

7 Play vs Render 15

7.1 The parameter blob 18

8 Mic array, mixer and patterns 19

9 Mic colour: FIR today, migrating to a 3-biquad IIR cascade 21

9.1 Where the colour data comes from 21

9.2 The FIR stage today 21

9.3 The IIR cascade being built (planned / in progress) 22

10 Parameters 23

11 Known issues and roadmap 24

11.1 Known issues and surprises 24

11.2 Roadmap 24

Appendix Sources 26

Figures are numbered in order; the list with one line each is in the figure captions. (In Word, right-click the contents and choose Update Field to refresh page numbers.)

1 What this document covers

This document explains how the 2011 Room Reverb software works, from the browser tabs down to the audio engine. It is written for someone who has not read the code. It describes the code as it stood in the snapshot, not as it is meant to become. Planned work is shown separately and tagged, so it is never mixed up with what runs today.

> **Snapshot used**
>
> Code snapshot taken on 28 Sep 2026, about 09:05 BST (byte-identical to the 27 Sep 2026 snapshot). The live folder on the PC (`Desktop\2011 Room Reverb\software`) is being edited right now: Grok Build started converting the Play mic colour stage from FIR to a 3-biquad IIR cascade at 09:33 BST (new `mic_colour/sos_fit.py` and `sos_table.py`, and changes to `live_bank.py`, `colour.py`, `modes.py`, `HOOKS.md` and `engine/live_stereo.c`). Those edits came after the snapshot and are not documented as final. Section 9 shows the colour stage as FIR today, migrating to IIR.

Every factual statement carries a tag:

- VERIFIED means I checked it in the code, or by running the code (for example with the server's default parameters).
- ASSUMED means it is inferred, taken from a design note, or not tested by listening.
- PLANNED means planned or in progress. It is not in the snapshot.

File paths are relative to `software\` unless they say otherwise. Code names are set in `Consolas`. All numbers for the "default room" come from running the real code with the server defaults VERIFIED.

How to read the diagrams:

- Blue means the browser/UI, green means Python, orange means DSP/audio, and grey means data/files.
- Red marks a known problem or a stage whose placement is disputed.
- Purple dashed boxes are planned or in progress.

2 Big picture: architecture

The app is a local web app. `Launch Tweak App.bat` creates a `.venv` on first run, installs `requirements.txt`, opens the browser and runs `python app_server.py`. That starts a **FastAPI** server under **uvicorn** on `127.0.0.1:7860` VERIFIED. (`tweak_app.py` is an older Gradio UI that the launcher no longer starts VERIFIED.) The server has 12 API routes plus a `/static` mount. All traffic is plain HTTP JSON (and one multipart upload); there is no WebSocket VERIFIED.

There are two ways to hear the room:

- **Play** (live): the browser runs the reverb itself. It uses an AudioWorklet (`fdn_worklet.js`) that calls a WebAssembly engine (`static/wasm/live_stereo.wasm`, built from `engine/live_stereo.c`). Python only computes the room parameters and sends them to the browser as a float "blob" VERIFIED.
- **Render** (offline): the browser uploads a WAV file. Python builds impulse responses with numpy (`render_pipeline.py` + `engine/`), convolves the file and writes `renders/out.wav`, which then plays in an `<audio>` element VERIFIED.

Both paths share one Python core: the geometry (`image_geometry.py`), the 32-image voice table, the FDN delay/diffusion design (`engine/fdn_stub.py`) and the mic colour designer (`mic_colour/`). The engines themselves are separate implementations (C/WASM vs numpy). Section 7 shows why the two can sound different.

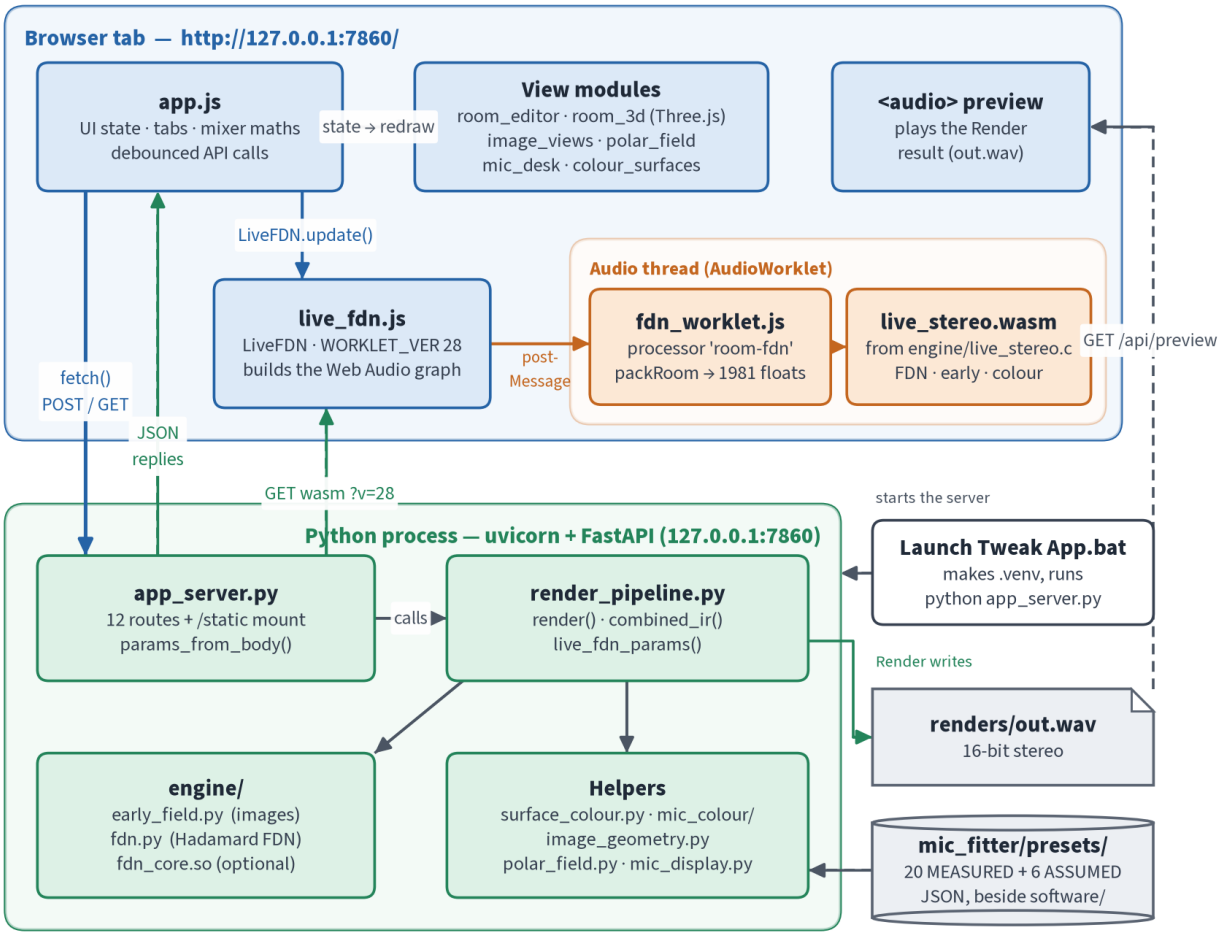


Figure 1. Architecture: the browser (UI, worklet, WASM engine) and the local Python server, with the shared core.

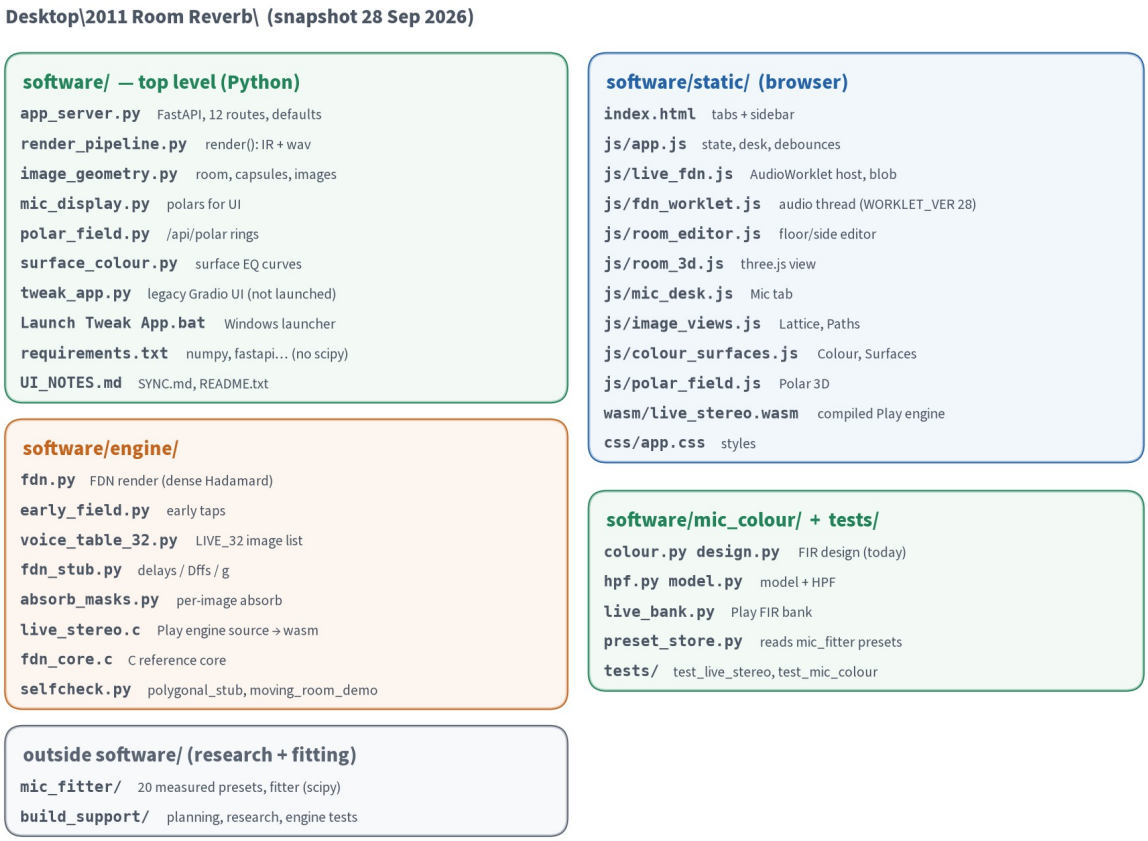


Figure 2. Folder map of the software, with the two research folders beside it.

3 The browser app

3.1 Tabs and sidebar

`static/index.html` has one row of tabs for the main view and a sidebar of collapsible panels. `syncSidebar()` in `app.js` shows or hides panels to match the tab VERIFIED:

- Room and Advanced are always shown.
- Listen appears with Mic, Polar 3D, Lattice and Paths.
- The Mix panel appears with Mix, Lattice, Paths, Colour, Surfaces and Polar 3D.
- Preview appears everywhere except Mic.
- The Hadamard size control lives in the Mix panel, so it is hidden on Floor and Mic.

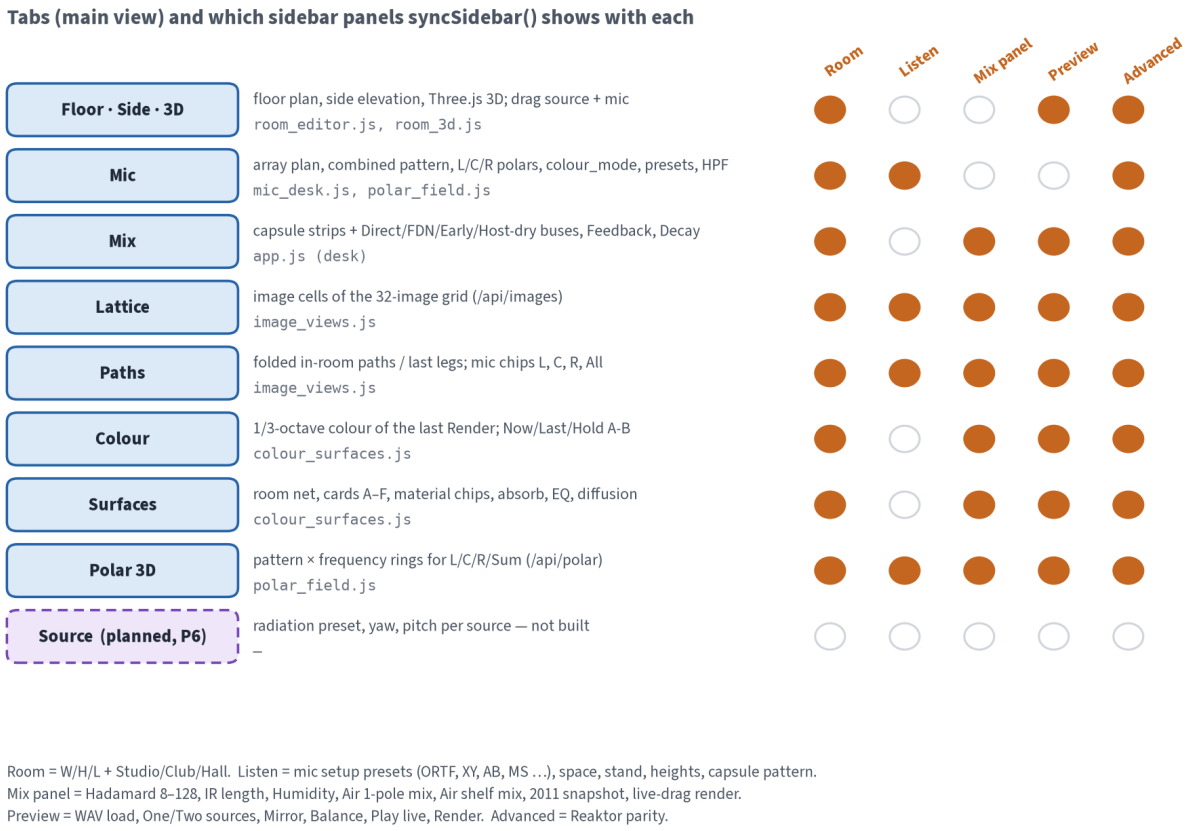


Figure 3. UI tab map: what each tab shows, the JS module behind it, and which sidebar panels appear with it.

3.2 How the UI talks to Python

Each control writes into one client state object in `app.js`. Changes are debounced before anything is sent VERIFIED:

- **Play parameters** (`POST /api/live_params`): 45 ms while playing. When idle and the decay lock is off, a decay preview is sent at 200 ms.
- **Image views** (`/api/images`): 280 ms, only when Lattice or Paths is visible.
- **Polars** (`/api/polar`): 120 ms, only on Mic or Polar 3D.
- **Live-drag render**: 420 ms after a drag, if enabled.
- The Colour tab has no request of its own. It updates from the reply to Render.

Five routes exist but the UI never calls them: `/api/defaults`, `/api/geometry`, `/api/mic_presets/{id}`, `/api/surfaces` and `/api/colour` VERIFIED. The most important consequence is that the client keeps its own copy of the defaults (see Known issues).

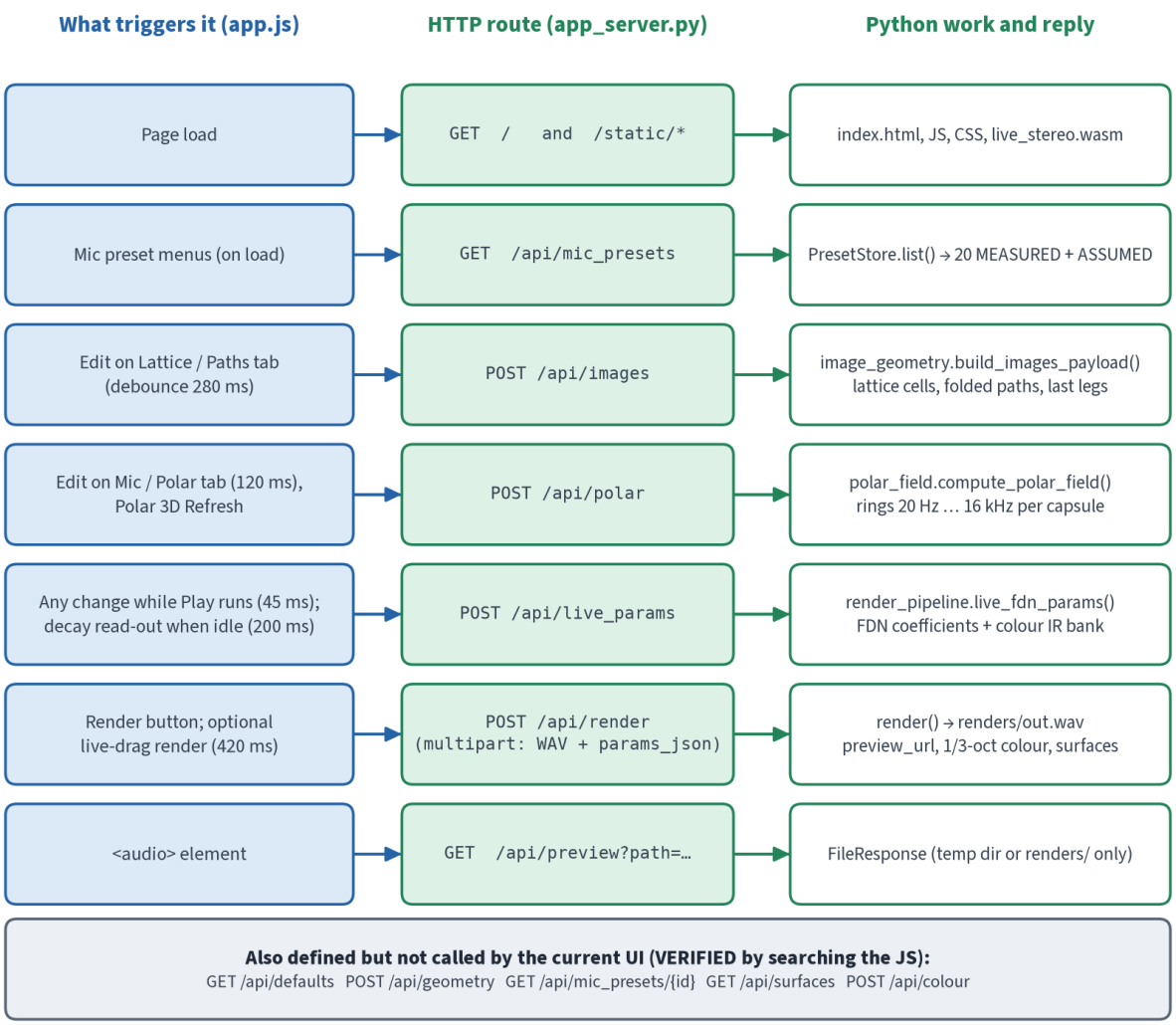


Figure 4. API routes: what triggers each call, and the Python work behind it. Grey routes are unused by the UI.

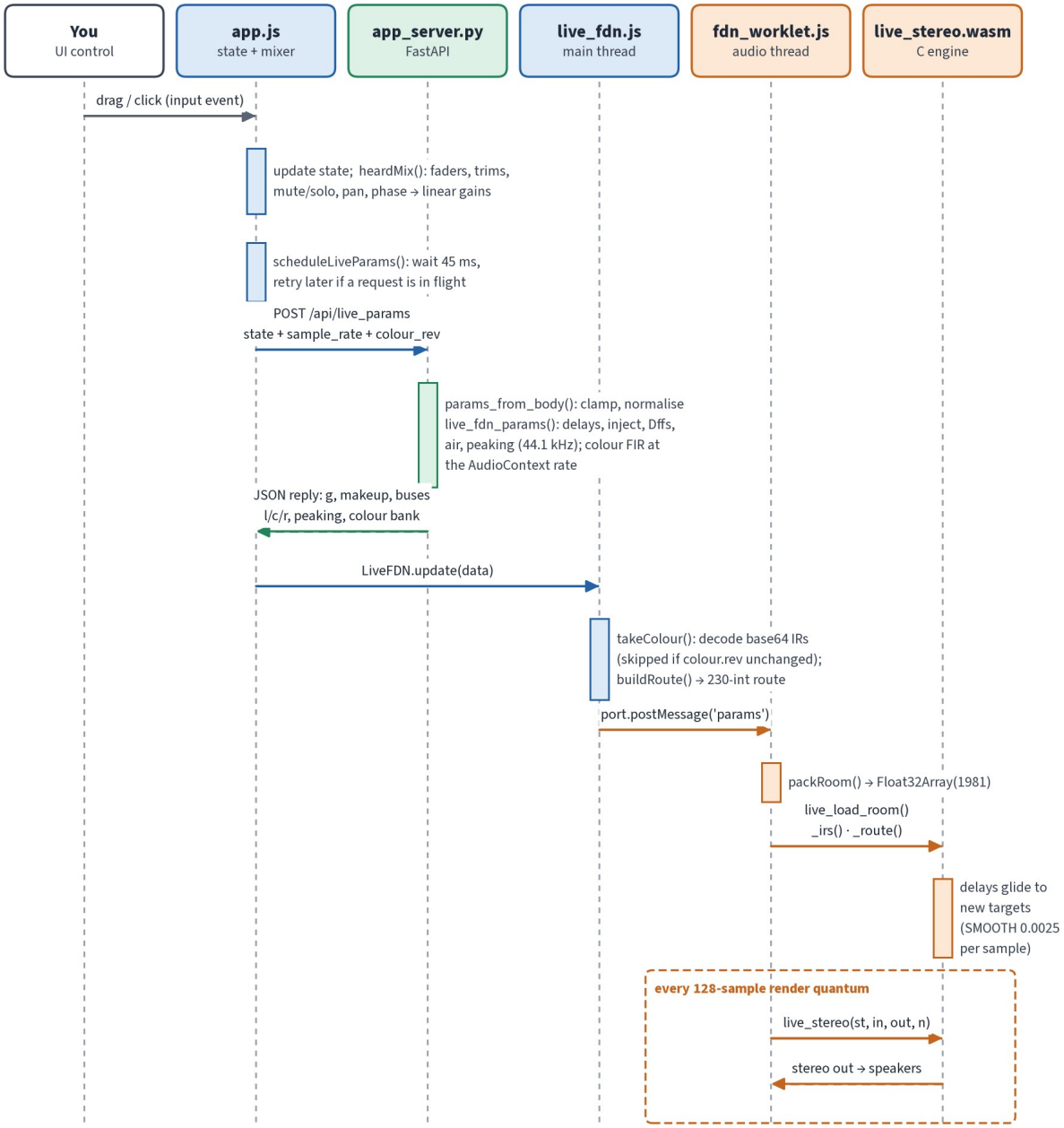


Figure 5. From a UI control to the WASM engine: debounce, /api/live_params, the blob, then postMessage to the audio thread.

4 The room model

4.1 Geometry and coordinates

The room is a shoebox. X is width (left to right), Y is height (floor to ceiling) and Z is length (back to front). The origin is a floor corner VERIFIED. The default room is 6 × 3 × 8 m. The source is at (3, 1.5, 4.75) and the mic stand at (3, 1.5, 1.5), so the direct path is 3.25 m, or 9.56 ms VERIFIED. The walls carry letters that the Surfaces tab uses: A left (−X), B front (+Z), C right (+X), D back (−Z), E floor, F ceiling.

The mic stand holds three capsules, L, C and R, spaced by `space` and aimed by a yaw angle. A capsule's look vector is (−sin yaw, 0, cos yaw) VERIFIED. The ORTF preset sets the yaws to ∓55°. With that formula, the L capsule (at lower X) points toward +X, so the pair is **crossed** compared with the floor plan (see Known issues, item K1).

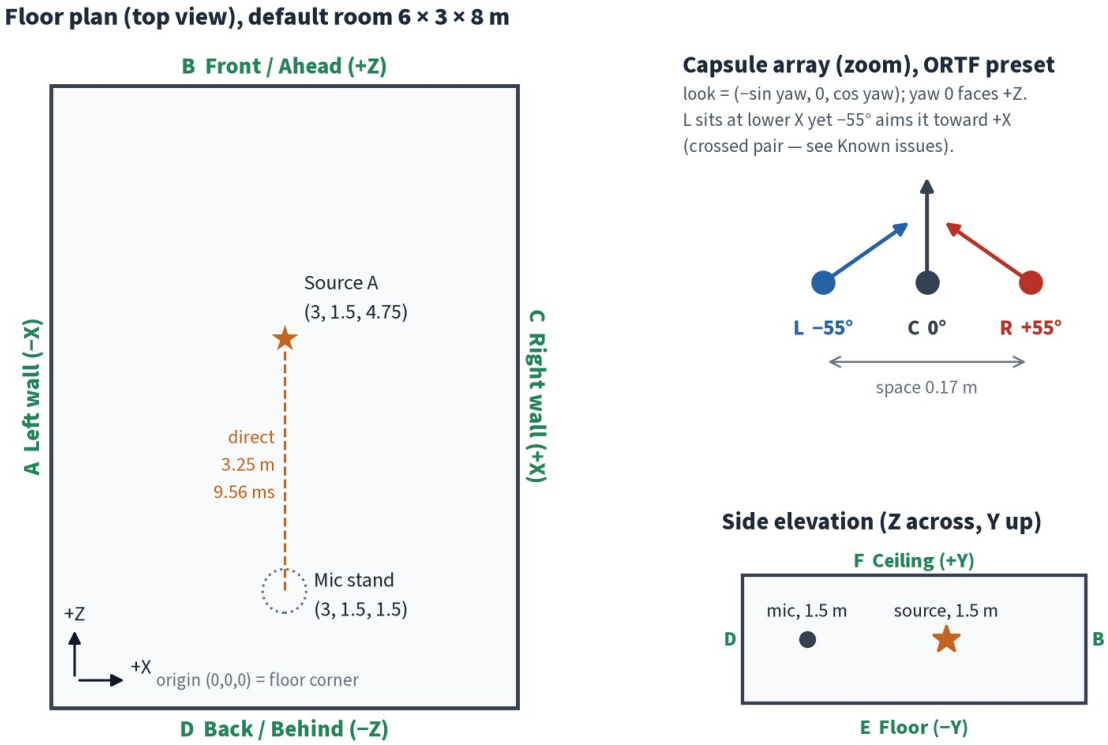


Figure 6. Default room: floor plan, capsule zoom (ORTF) and side elevation.

4.2 Image sources and the 32 voices

Reflections are modelled with the image-source method. Each image n = (n_x, n_y, n_z) is a mirror copy of the source; odd mirror counts flip its polarity. Play always uses the fixed **LIVE_32** table (`engine/voice_table_32.py`): 32 lines, each fed by one image VERIFIED. It contains 29 unique images. Three are used twice: (0,1,1), (0,0,1) and (−1,−2,0). The image (−1,−1,0) is missing, and voice 3 (index 2) is the direct sound VERIFIED. Render can size the grid by `hadamard_n` (8–128).

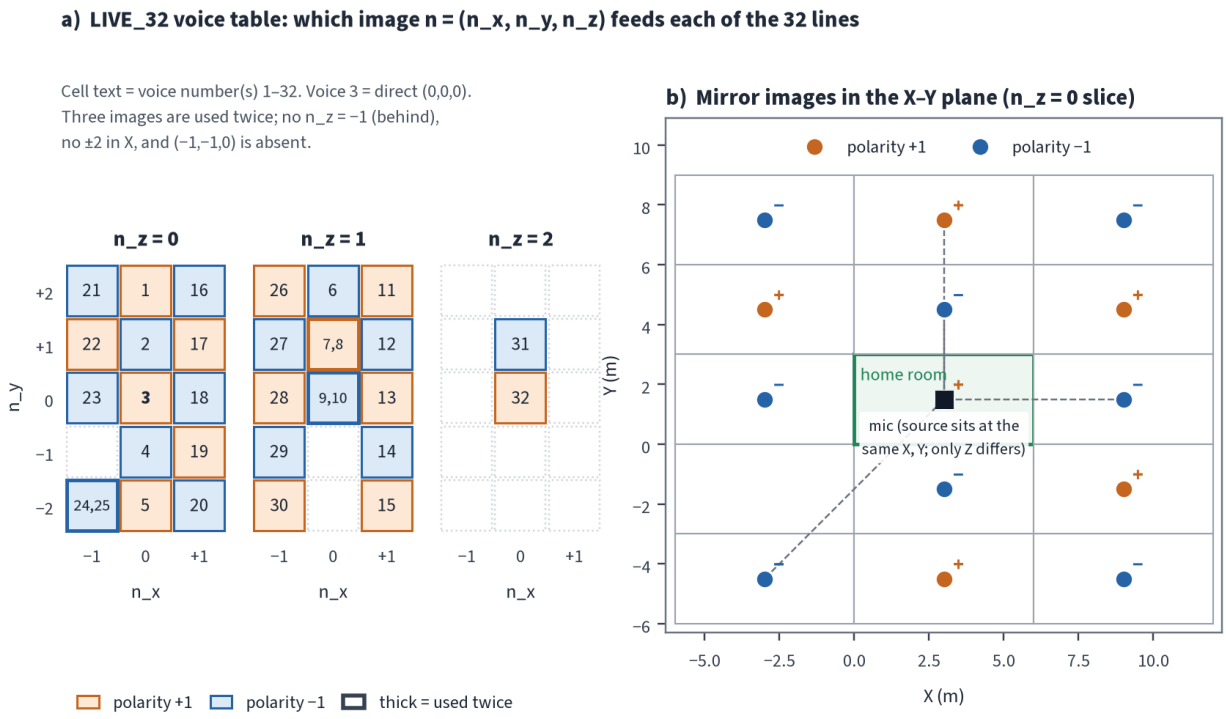


Figure 7. a) Which image feeds each of the 32 lines. b) The X–Y mirror-image slice around the home room.

Each image's distance becomes the delay of its FDN line; the prime-number proposal in `fdn_stub.py` is only a fallback. Because the default source and mic are both centred in X, mirror pairs have equal distances, so the 32 line delays contain only **12 distinct values** VERIFIED. This matters for tail density (item K3).

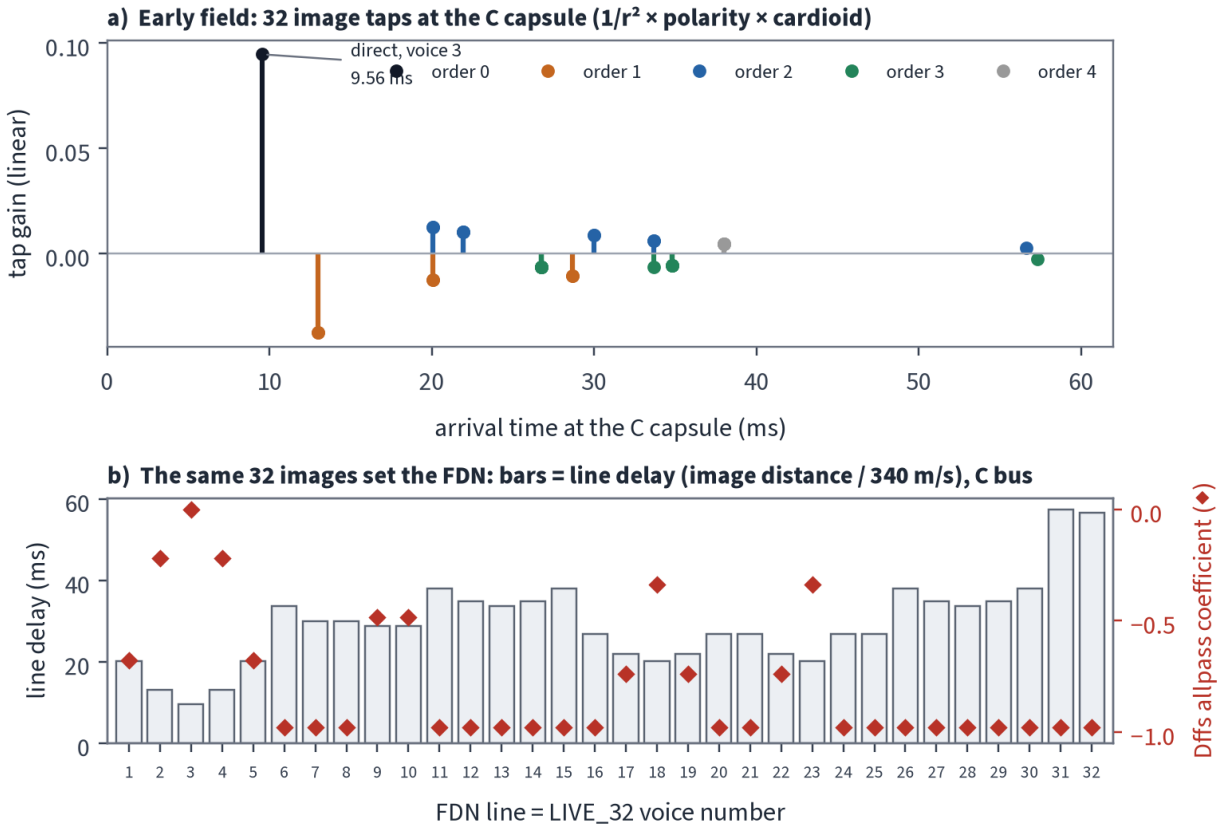


Figure 8. Default room: echogram of the 32 taps at the C capsule, with each line's delay and diffusion (Dffs).

5 The FDN engine

The late tail is a 32-line feedback delay network (FDN). Every sample, each line's output is passed through a Schroeder allpass (the per-line diffusion, "Dffs"). The 32 outputs are then mixed by a Hadamard matrix, scaled by the loop gain g, and written back with the new input VERIFIED.

- **Mixing matrix:** a 32×32 Sylvester Hadamard matrix of ±1 values. Play applies it with a fast Walsh–Hadamard transform (FWHT, 5 butterfly stages). Render multiplies by the dense matrix in numpy VERIFIED.
- **Loop gain:** g = 1/32 with the unnormalised matrix. In the default room the makeup gain is 4.92, the mean delay is 29.3 ms, and the decay estimate is 0.117 s with the lock off VERIFIED.
- **Dffs:** in the default room many lines have Dffs −0.98 VERIFIED. Positive Dffs makes the loop unstable, and the server does not clamp `diffusion_master` (item K6).

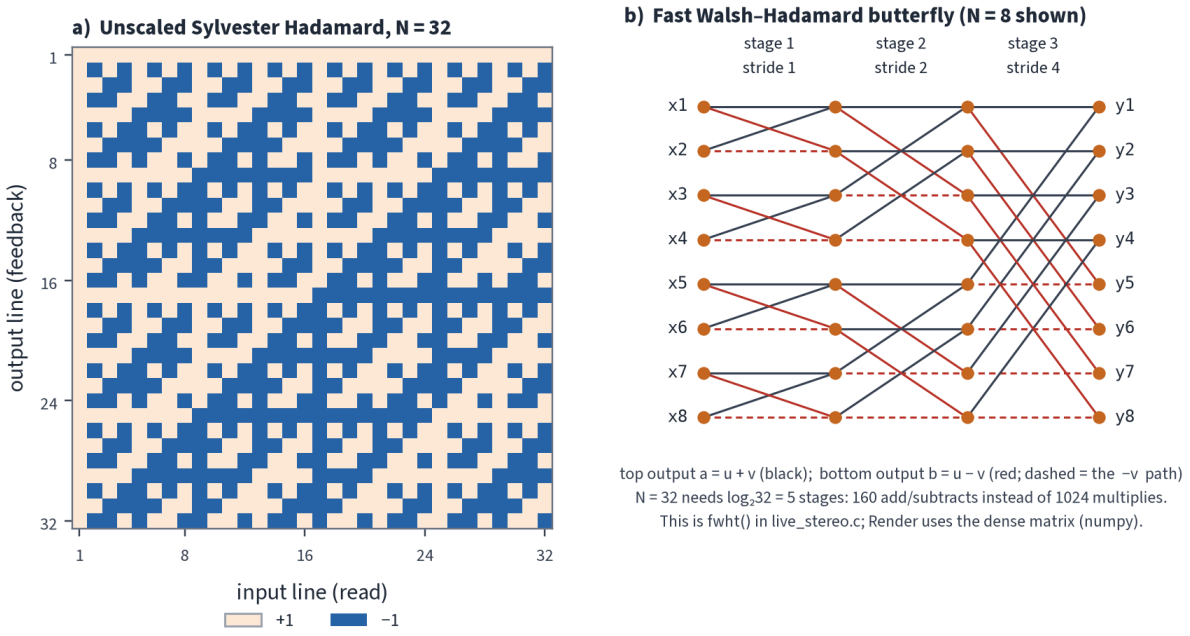


Figure 9. Left: the 32×32 Hadamard matrix. Right: the FWHT butterfly for N = 8, the same idea Play uses at N = 32.

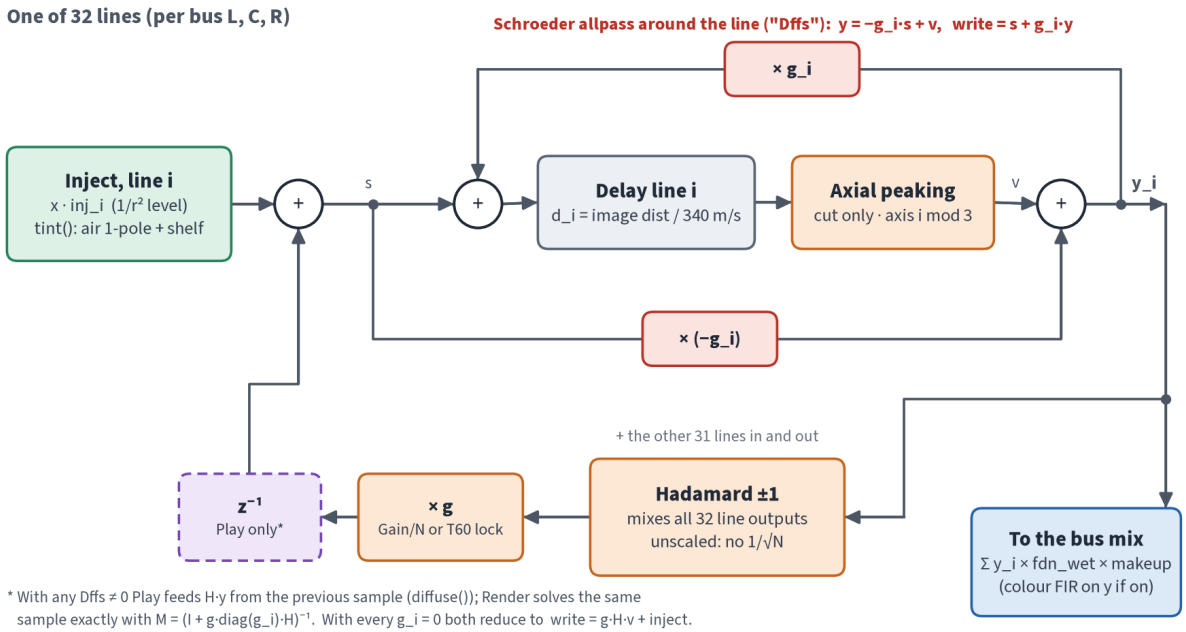


Figure 10. One FDN line. The z^{-1} on the feedback into the allpass exists only in Play.

Loop semantics differ between the two engines VERIFIED:

- Render solves the allpass and the matrix in the same sample: yv = M(reads − dffs·inj), s = inj + g·H·yv, write = s + gi·yv.
- When any Dffs is non-zero, Play's `diffuse()` uses the previous sample's feedback (a one-sample z^{-1}).
- The roadmap's P0.4 must pick one. Engine finding F5 shows that a same-sample solve in Play costs +0.16–0.27 ms per block.

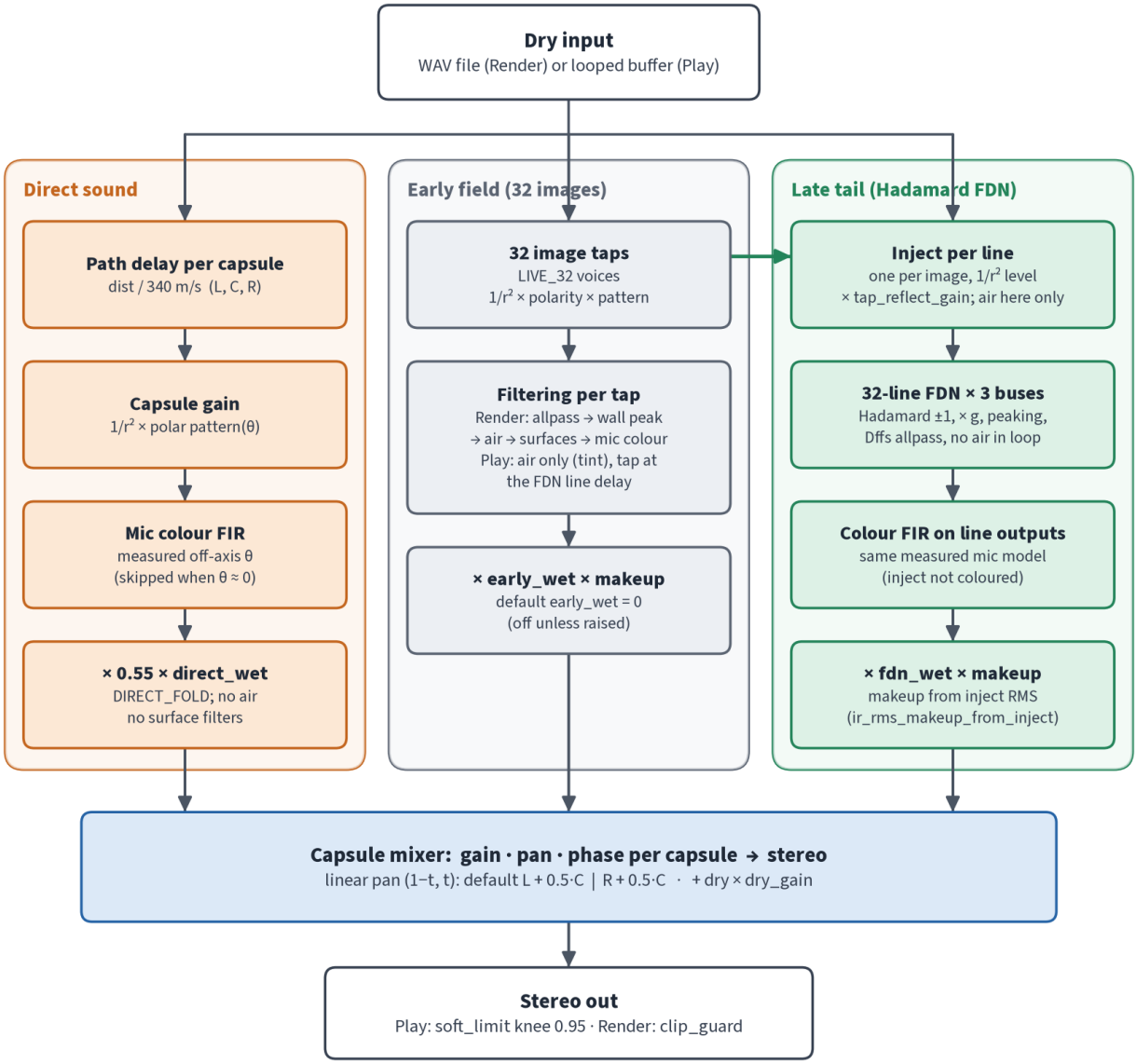


Figure 11. Signal flow: the Direct, Early and FDN paths, then the capsule mixer and the output.

6 Per-tap processing: early, air and surfaces

Before a reflection reaches the mic, it is shaped by several filters. The roadmap locks a target order: **source → air → surfaces → mic colour**. Direct sound gets no air, and the FDN return gets no air VERIFIED, from BUILD_ROADMAP. Today's code does not follow that order exactly. Render inserts an allpass stage and a wall-peaking cut that are not in the locked list, and Play has no per-hit surface cascade yet.

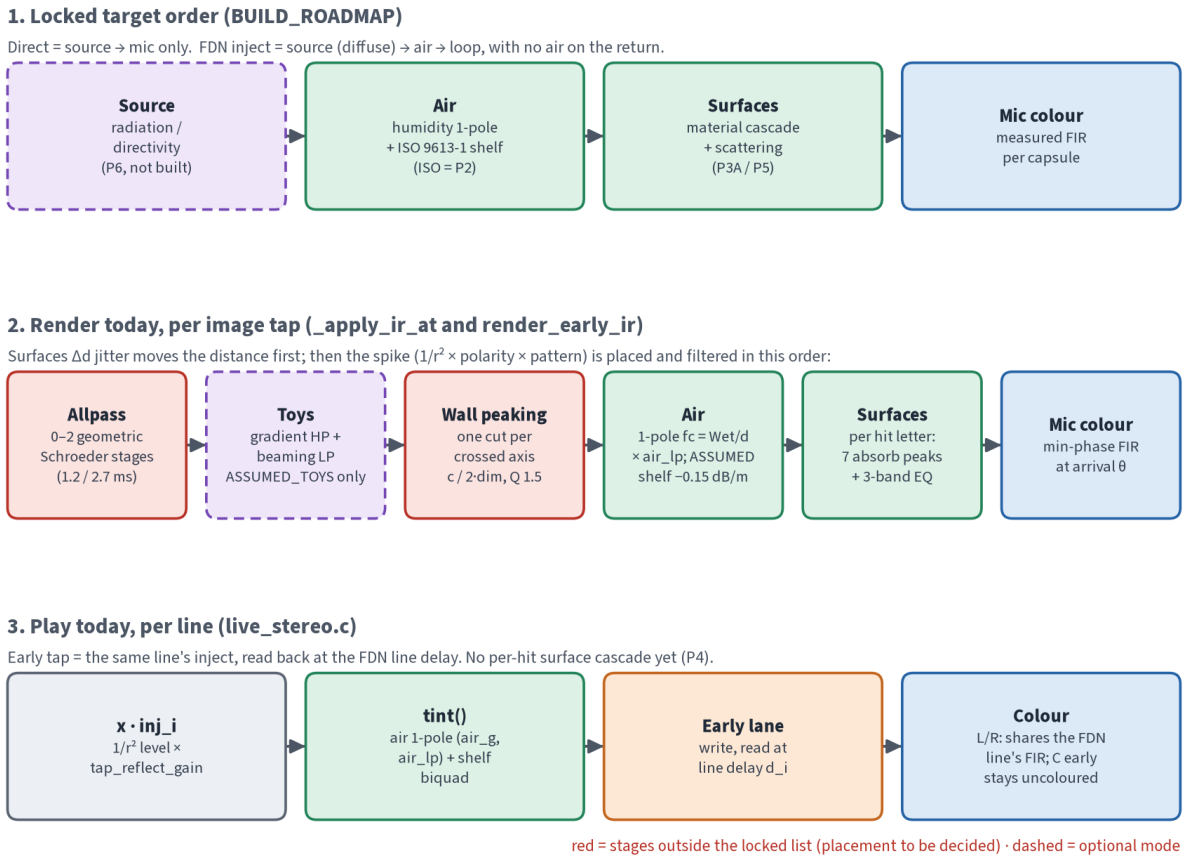


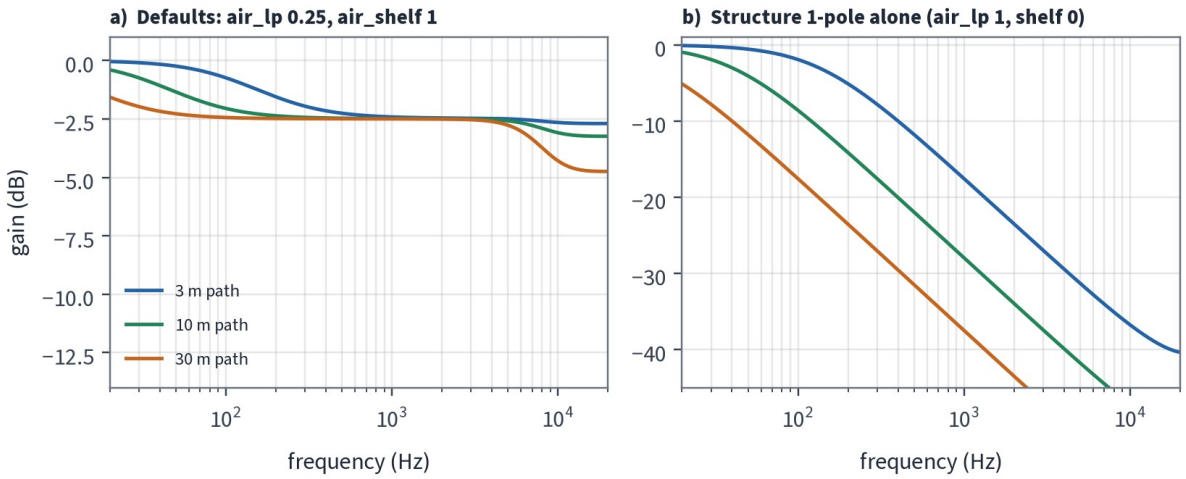
Figure 12. Locked target order vs what Render and Play do today. Red stages are outside the locked list.

6.1 Air

Air has two layers VERIFIED:

- A "Structure" 1-pole low-pass with cutoff fc = Wet / distance, where Wet = 100 + 600·humidity. It is blended in by `air_lp` (default 0.25).
- An RBJ high shelf of −0.15 dB per metre × humidity, capped at −12 dB, with a corner at 0.35 × Nyquist. This shelf is marked ASSUMED in the code; it is not ISO 9613-1.

The 1-pole corner is very low: at humidity 0.5 it is about 133 Hz at 3 m. At a mix of 0.25, that becomes an almost broadband −2.5 dB step rather than an HF roll-off. The units may have been misread from the original Reaktor graph ASSUMED (item K2). P2 replaces the shelf with an ISO 9613-1 fit.

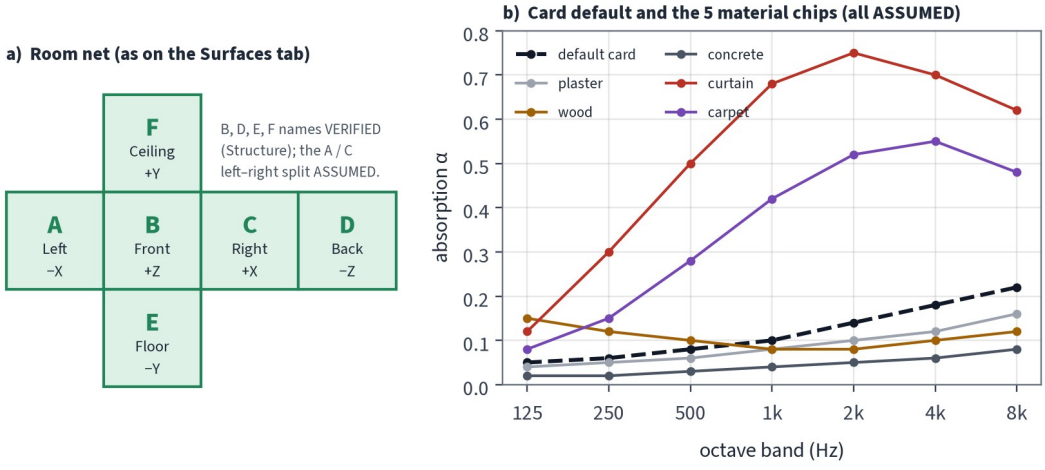


Humidity 0.5 → Wet = 100 + 0.5·600 = 400; 1-pole fc = Wet / distance (VERIFIED, Structure). Shelf: −0.15 dB/m × d × humidity, capped at −12 dB, corner 0.35 × Nyquist (ASSUMED RBJ high shelf, not ISO 9613-1). Computed with apply_air_absorption() at 44.1 kHz.

Figure 13. Air response at 3, 10 and 30 m (humidity 0.5 for illustration): defaults vs the 1-pole alone.

6.2 Surfaces

Each wall has a card with absorption values (7 peaking bands) and a 3-band EQ. It also carries a `diffusion` value that currently feeds **both** the Surfaces Δd jitter and the Structure Dffs (item K7, P0.3). In Render, each reflection is filtered once for each wall it hits. Play only has one early tap per line and applies `tap_reflect_gain` as a scalar VERIFIED.



Each band becomes a peaking cut (Q 0.9) of 20·log10 √(1 − α) dB per hit, plus the card's low shelf, peak and high shelf. FDN inject is scaled by tap_reflect_gain = Π √(1 − mean α) over the surfaces hit.

Figure 14. Room net with the wall letters, and absorption curves for the default card and the five material kits.

7 Play vs Render

Play and Render build the same room from the same Python core, but they run it in different engines. The table lists the differences that matter for sound.

Table 1. Play vs Render at a glance.

Aspect	Play (live, browser)	Render (offline, Python)
Engine	`live_stereo.c` compiled to WASM, in an AudioWorklet VERIFIED	numpy in `render_pipeline.py` + `engine/fdn.py` VERIFIED
Images / lines	Fixed LIVE_32, one early tap per line VERIFIED	Grid sized by `hadamard_n` 8–128 VERIFIED
Hadamard	FWHT, N = 32 VERIFIED	Dense matrix product VERIFIED
Loop with Dffs	One-sample z^{-1} in `diffuse()` VERIFIED	Same-sample implicit solve VERIFIED
Early field	Same line's inject, read at the line delay; the C bus early is uncoloured VERIFIED	Per image, L/R capsules only (no C), only if `early_wet` > 0 VERIFIED
Per-tap filters	air 1-pole + shelf (`tint()`) VERIFIED	allpass, toys, wall peaking, air, surfaces, colour VERIFIED
Mic colour	Partitioned FFT FIR, 128/256; only if the colour sample rate equals the AudioContext rate VERIFIED	Min-phase FIR per tap in Python VERIFIED
Sources	One: x = ½(L+R). Two: xA = L·gain_a, xB = R·gain_b; each tank gets 0.5·x VERIFIED	Still, two-source (IR per source) or moving path (Hann OLA) VERIFIED
Delays / coefficients	Rounded and designed at the server rate, not the Play rate (F3/F4) VERIFIED	At the file rate VERIFIED
Latency / CPU	No extra latency at a 128 quantum; 1.17 ms (1 source) / 1.77 ms (2 sources) per 128-sample block, against a 2.667 ms budget at 48 kHz VERIFIED	Offline

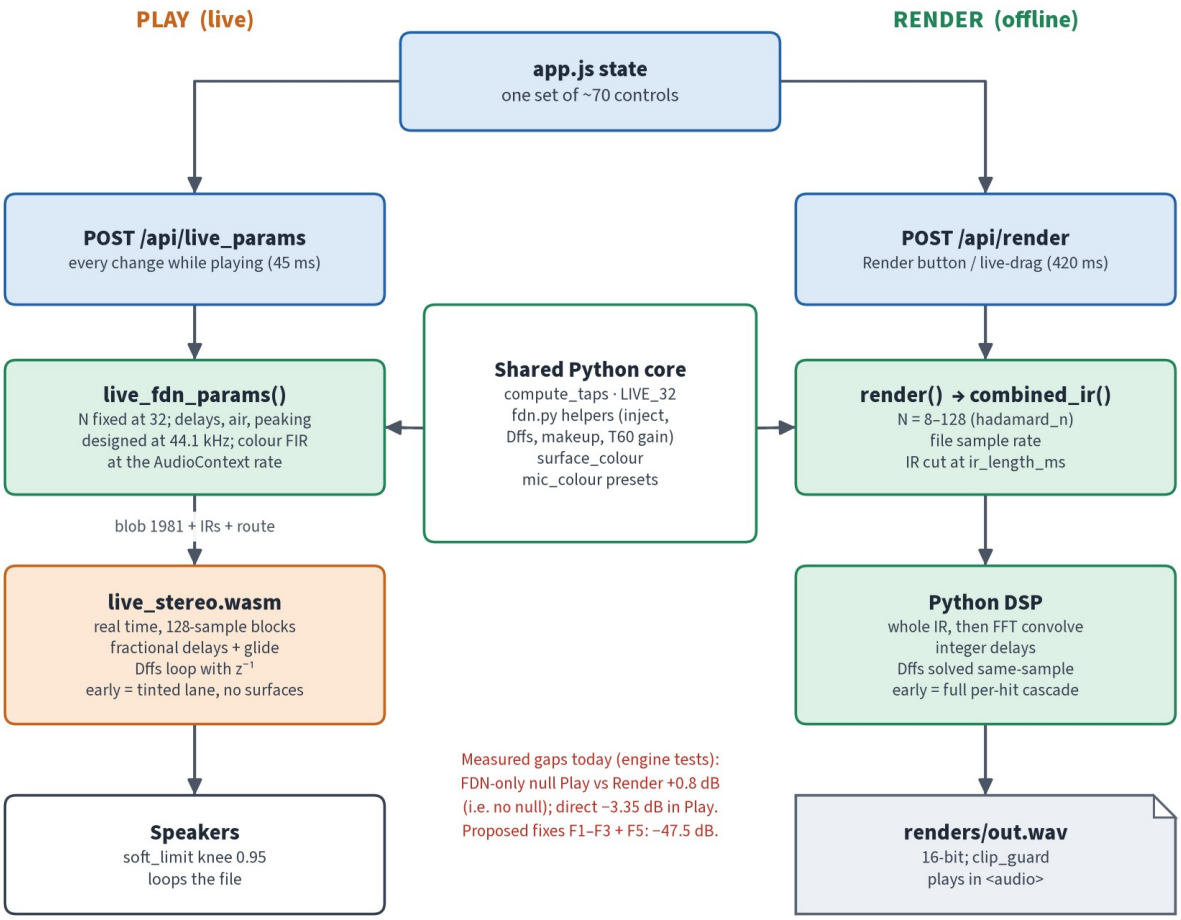


Figure 15. The two lanes and the shared Python core.

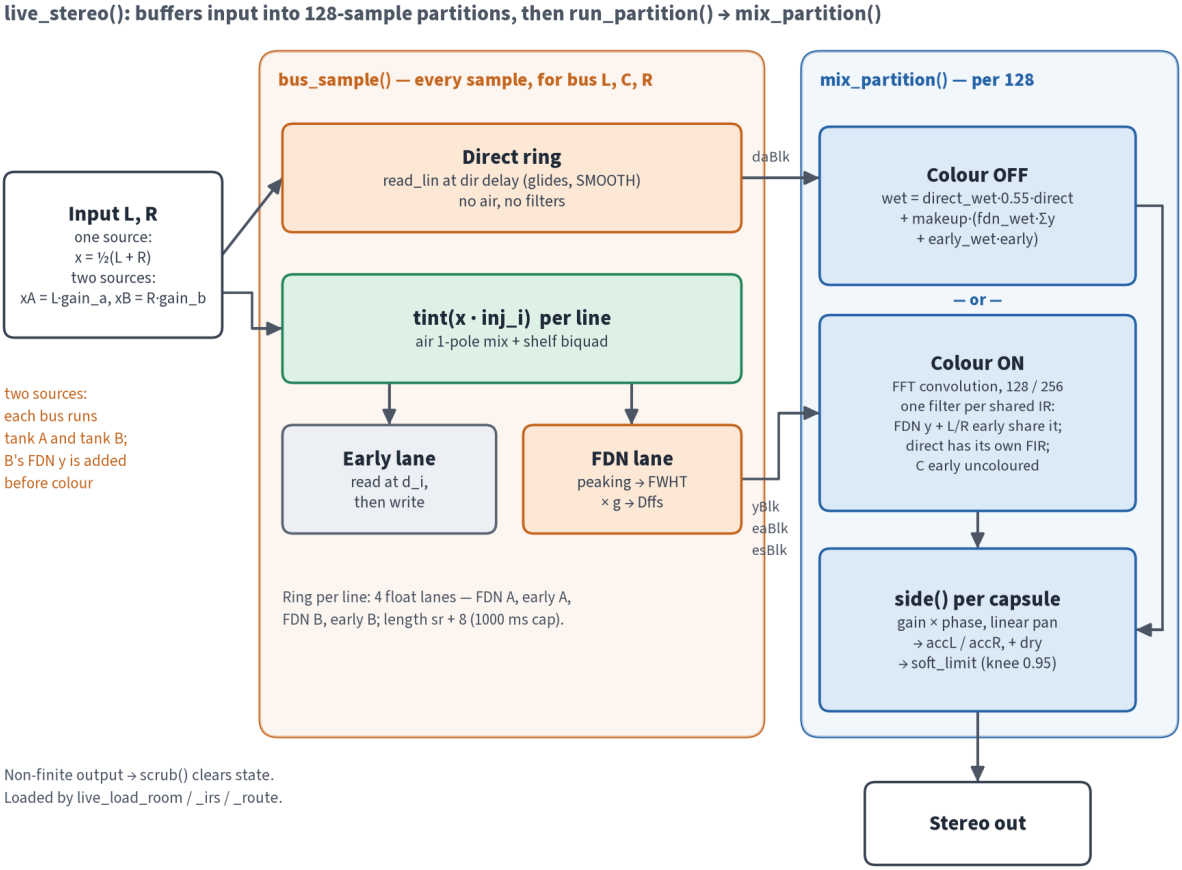


Figure 16. Inside the Play engine: per-sample bus work, then per-partition mixing and colour.

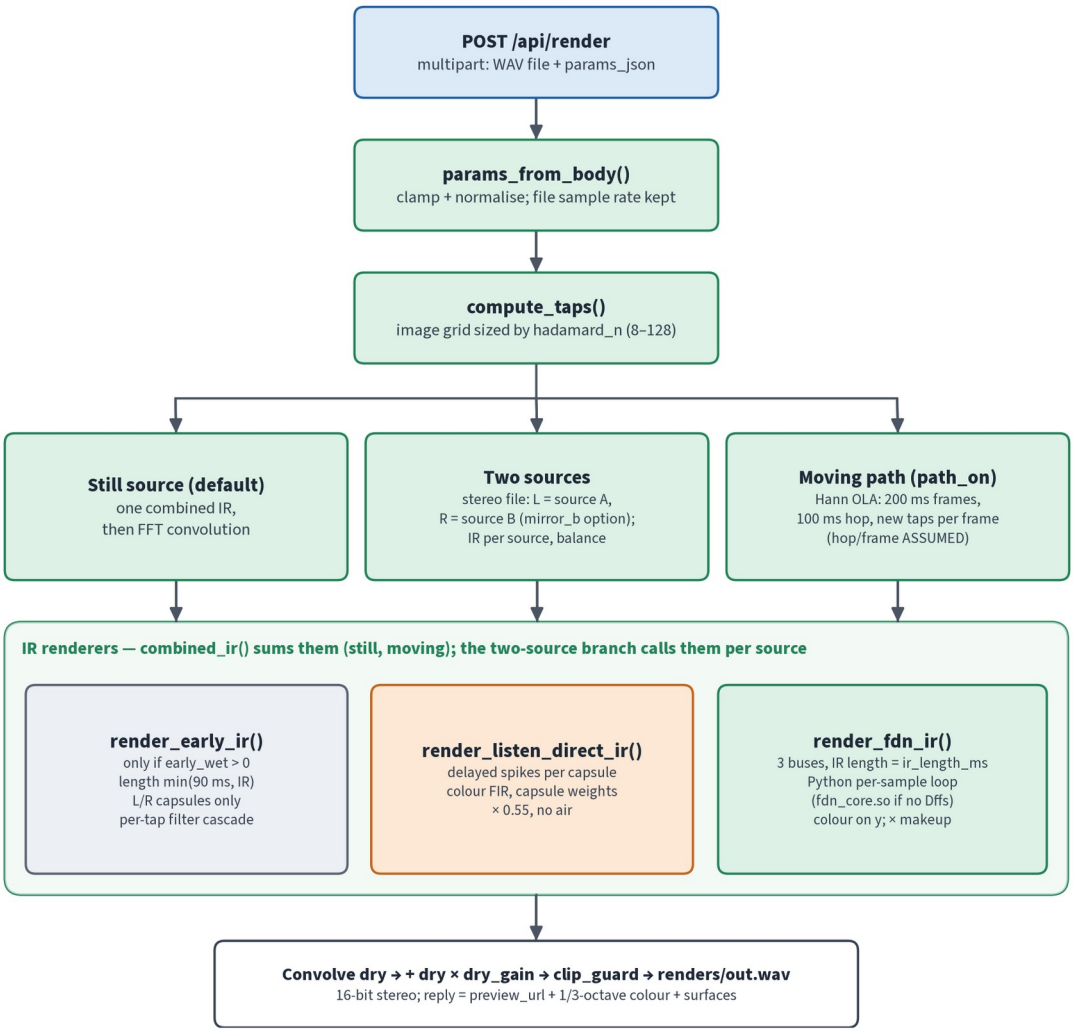


Figure 17. Render: request, taps, the three branches, the IR renderers and the output.

7.1 The parameter blob

Python packs the Play room into a 1981-float blob plus a 230-int route table VERIFIED:

- **Header** (16 floats): magic, g, fdn_wet, direct_wet, early_wet, dry_gain, makeup, gain_a, gain_b, two, colour_on, mono, direct_index, max_delay_ms, snap, and one unused slot.
- **Sides:** six sides of 323 floats, packed in the order L-A, L-B, C-A, C-B, R-A, R-B.
- **Versioning:** `WORKLET_VER` is 28. P1 plans a versioned v2 blob with a section table.

Room blob: Float32Array(1981) built by packRoom() and read by live_load_room() (magic = 1, no version field)

Header 0–15	Mic L/C/R 16–24	Peaking ×3 25–42	L · src A 43–365	L · src B 366–688	C · src A 689–1011	C · src B 1012–1334	R · src A 1335–1657	R · src B 1658–1980

Zoom 1 — the orange Header, 16 floats (float index under each cell)

magic =1	g	fdn _wet	direct _wet	early _wet	dry _gain	makeup	gain _a	gain _b	two src	colour _on	mono	direct index	max delay ms	snap	unused (0)
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Zoom 2 — one grey bus side, 323 floats (pushSide), repeated for L, C, R × source A, B

delay_ms 32	inject 32	dffs 32	air_g 32	air _lp 1	shelf biquad per line 32 × (on, b0, b1, b2, a1, a2)	dir ms 1	dir gain 1

dir_ms = delay_ms[direct_index] (line 2 by default). Mic block: gain, pan, phase for L, C, R. Peaking: (on, b0, b1, b2, a1, a2) for the X, Y, Z axes.

Colour route: Int32Array(230) (live_load_route)

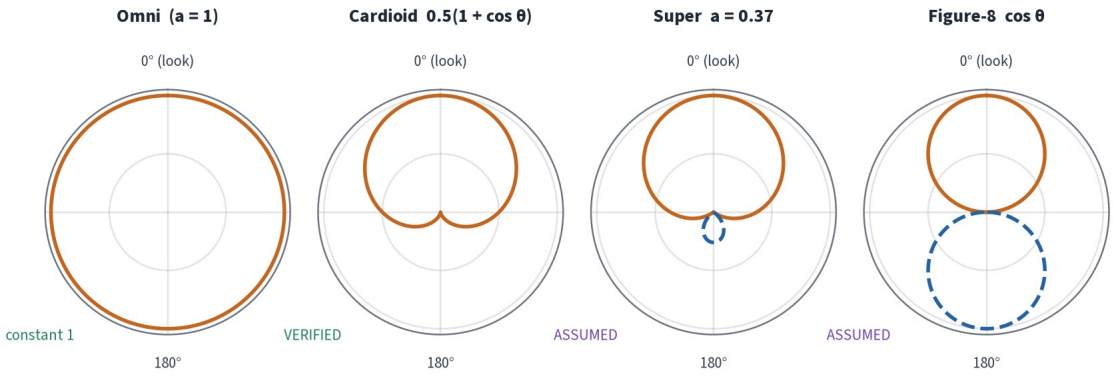
fdn IR index [3][32]	early L/R [2][32]	early B [2][32]	dir [3]	dir B [3]

Each entry picks a shared colour IR: ≥ 0 = IR number, −1 = straight wire (no filter), < −1 = silent.

Figure 18. Blob layout: header, one side, and the route table.

8 Mic array, mixer and patterns

Each capsule has a first-order pattern: omni, cardioid, supercardioid (0.37) or figure-8 VERIFIED. The combined gain of a tap depends on the angle between the capsule's look vector and the arrival direction.



Solid orange = in-phase lobe; dashed blue = rear lobe with inverted polarity (signed gain is used on the taps).
Polar slider: 0 = omni (a = 1) → 0.5 = cardioid (a = 0.5) → 1 = super (a = 0.37), gain = a + (1 − a)·cos θ. θ = acos(look · û) in 3D.

Figure 19. Capsule polar patterns.

The Mix tab is a small desk VERIFIED:

- **Capsule strips (L, C, R):** trim ±12 dB, pan, phase (Ø), fader and mute. They have no solo.
- **Bus strips (Direct, FDN, Early, Host dry):** trim, fader, mute and solo.
- **Pan law:** linear (1−t, t). By default L+0.5C goes left and R+0.5C goes right.
- **Stereo mic:** links the L/R faders and forces the pans to ±1.

- **Natural:** links the Direct and FDN faders and trims.
- **Early is silent by default:** `early_wet` = 0 and `fader_early` = 0, so nothing reaches the early bus until you raise it (item K4).

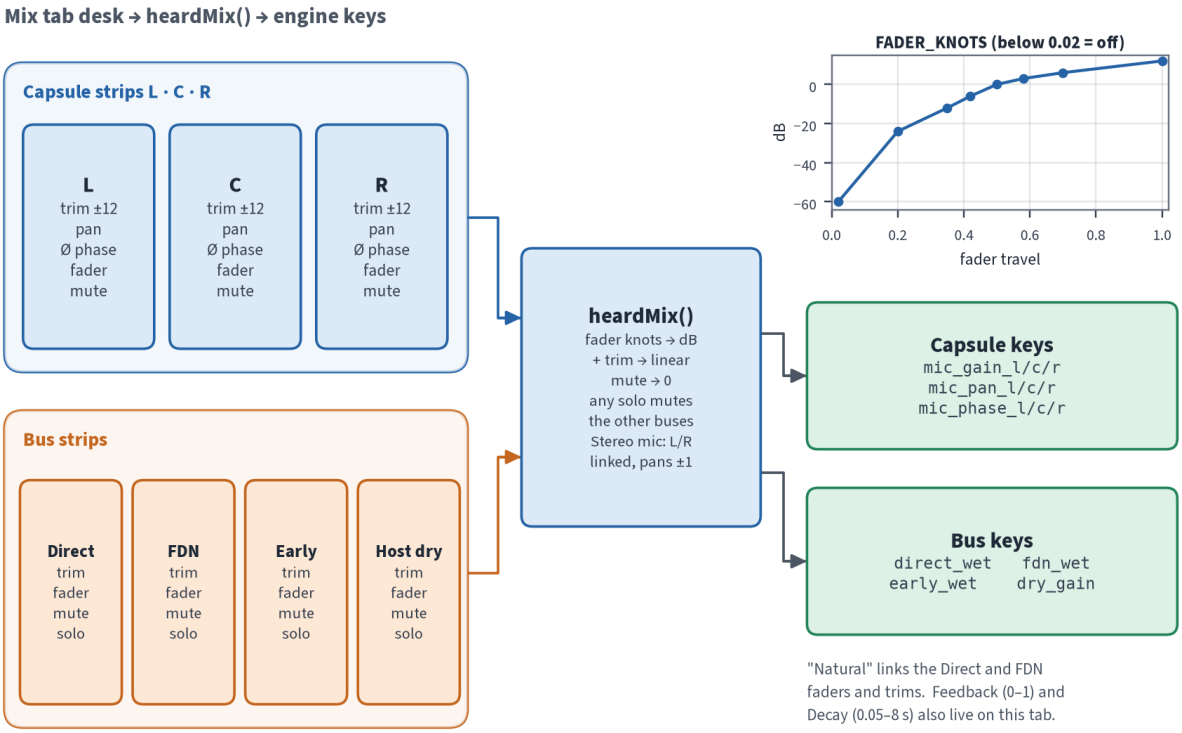


Figure 20. Mixer: desk controls → heardMix() → blob keys, with the fader curve inset.

9 Mic colour: FIR today, migrating to a 3-biquad IIR cascade

Status on 28 Sep 2026

Currently FIR, migrating to a 3-biquad IIR cascade (in progress by Grok Build). The snapshot contains only the FIR stage, so that is what this section documents as live. The IIR design below comes from the cheap_colour research. It is shown as planned/in progress, not as final.

9.1 Where the colour data comes from

`mic_fitter` (a separate folder) fits 20 **measured** microphones from the Franco et al. multi-angle IRs VERIFIED:

- **Measurement window:** 2 ms before and 80 ms after the arrival.
- **Model:** `dual_shelf_plus_soft_one_pole`, with 9 parameters.
- **Angle law:** w = ½(1−cos θ)^p.
- **Fitting:** differential evolution followed by least_squares, with 45° and 135° held out.
- **Output:** presets in schema v3, plus 6 ASSUMED proxies.
- **Accuracy:** hold-out RMSE ranges from 0.93 dB (DPA 4006) to 5.66 dB (MD441).

`mic_fitter` needs scipy. The software itself does not: its tests assert that scipy is never imported VERIFIED.

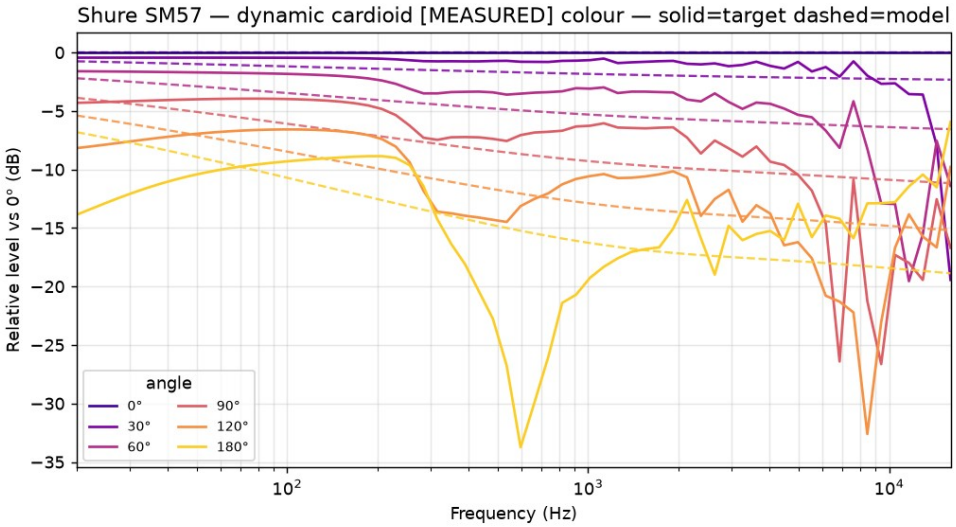


Figure 21. Example fitted colour (Shure SM57), reused from mic_fitter/plots.

9.2 The FIR stage today

`mic_colour/design.py` turns a preset's dB curve at the arrival angle into a real-cepstrum minimum-phase FIR of about 80 ms (1024–8192 taps). The mode can be MEASURED_SHELF (default), BYPASS or ASSUMED_TOYS VERIFIED.

- **Render:** applies the FIR in Python to each image tap and each FDN line output. The FDN inject is not coloured.
- **Play:** `live_bank.build_live_bank()` shares IRs by (preset, θ, sr, HPF) and sends them base64-encoded with a hash, so unchanged banks are not sent again. `live_stereo.c` convolves them with a uniformly partitioned FFT (128/256).

Colour is about 57 % of the Play block cost VERIFIED. The research also found that today's FIR misses its own target by 14.5–23 dB when the manufacturer HPF is on, for 7 mic families VERIFIED, from cheap_colour.

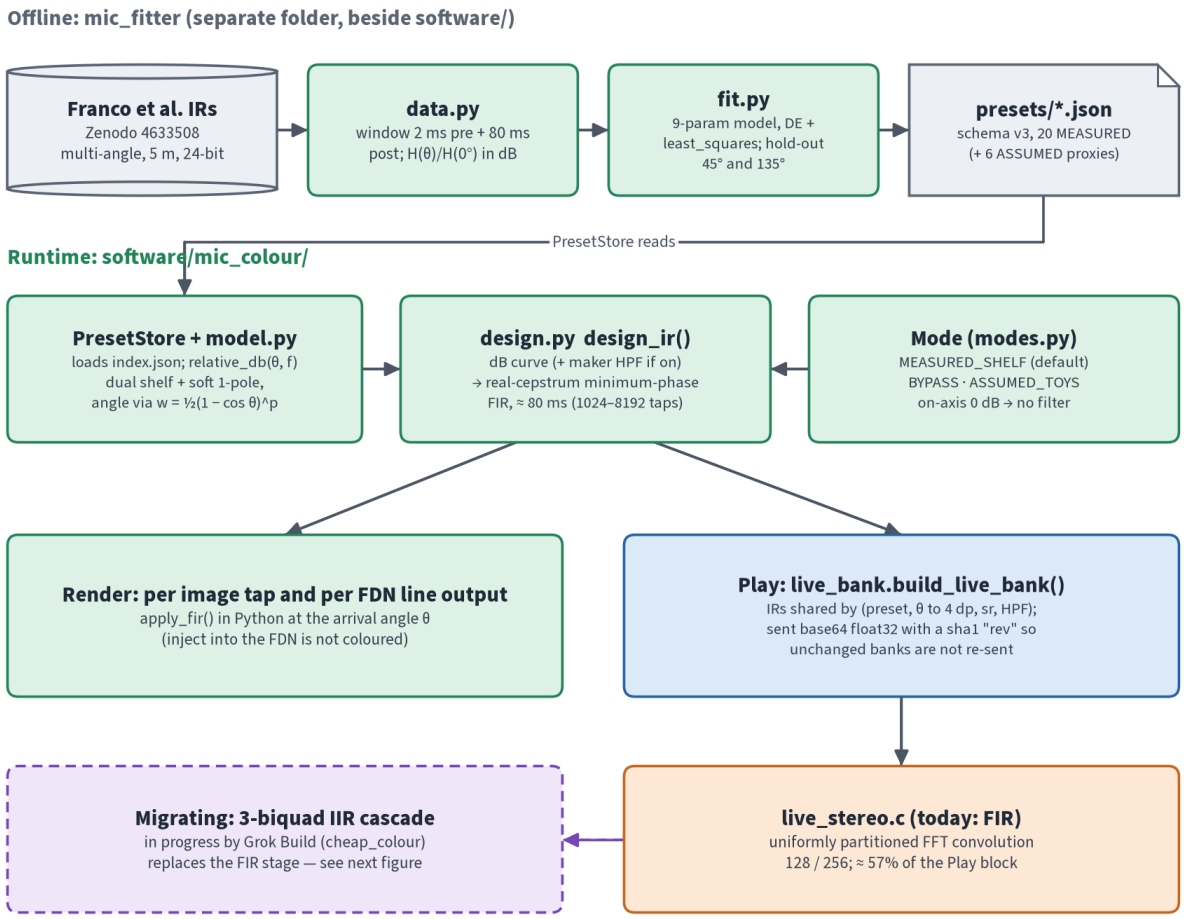


Figure 22. Mic colour today: offline fitting → runtime design → Render / Play FIR, with the migration marked.

9.3 The IIR cascade being built (planned / in progress)

The cheap_colour research replaces each FIR with **3 biquads**: a low shelf and two high shelves (RBJ, minimum phase). When the maker's HPF is on, exact Butterworth HPF sections are added PLANNED. The parameters (f, gain, Q) for each arrival angle come from a per-mic w-table. The table is built offline with 65–160 nodes (about 288 KB of JSON for 20 mics) and read by binary search plus linear interpolation.

Figures measured in the research VERIFIED, from cheap_colour:

- **Colour cost:** 0.662 → 0.081 ms per block.
- **1-source Play block:** 1.158 → 0.577 ms.
- **Error vs today's FIR:** max 0.46 dB, p95 0.24 dB.
- **Play render null:** −47.7 to −48.5 dB.

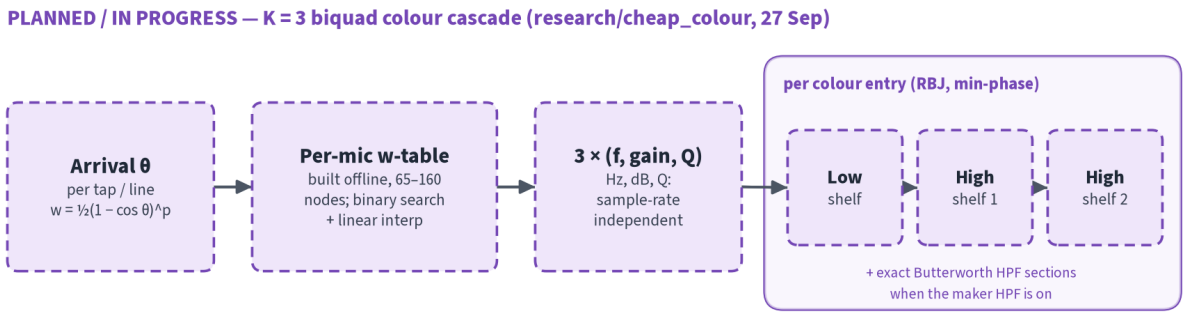


Figure 23. PLANNED / IN PROGRESS: the K = 3 biquad colour cascade from the cheap_colour research.

10 Parameters

The full list is in `parameters.csv` (copied next to this document, from `build_support/planning`) VERIFIED:

- 156 rows: 135 current and 21 pending.
- About 70 distinct current controls.
- 10 rows flagged CONFLICT and 13 flagged DUPLICATE.

The table below summarises the groups. See `PARAMETERS.md` for the meaning of each flag.

Table 2. Parameter groups (counts from parameters.csv).

Group	Rows	Examples / notes
Room	4	`W`, `H`, `L`, `link_heights`
Source A	3	`sx`, `sy`, `sz`
Source B	6	`two_sources`, `mirror_b`, `source_balance`, `sx_b`…
Capsules	16	`pattern_l`, `family_l`, `polar_l` … per capsule
Mic array	7	`mcx`, `mcz`, `my`, `space`, `diverg`, `toe_half_deg`, `car_omni`
Mic colour	10 + 1 pending	`colour_mode`, `mic_preset_l/c/r`, `mic_setup`, `preset_owns_pattern`
Mic toys	5	`mic_hp_on`, `mic_hp_hz`, `mic_response_hz`, `mic_proximity`, `capsule_mm`
Mix	13	`early_wet`, `fdn_wet`, `direct_wet`, `dry_gain`, `mic_gain_l`, `mic_pan_l` …
Mix desk (client)	38	`trim_*`, `fader_*`, `mute_*`, `phase_*`, solos, links
Tail	5	`fdn_g`, `decay_s`, `decay_lock`, `hadamard_n`, `diffusion_master`
Air	3 + 1 pending	`humidity`, `air_lp`, `air_shelf`; `air_climate` pending (P2)
Surfaces	13 + 5 pending	`surfaces[X].absorb[7]`, `.diffusion`, shelf EQ …
Scattering	5 pending	`scatter[7]`, `scatter_preset`, `scatter_amount`, `scatter_pulses`, `scatter_seed` (P5)
Source radiation	9 pending	`src_rad.preset`, `yaw_deg`, `pitch_deg` … per source (P6)
Paths	5	`path_on`, `path_preset`, `ellipse_rate`, `ellipse_rx`, `ellipse_rz`
Render	2	`ir_length_ms`, `reaktor_parity`
System	2	`sample_rate`, `colour_rev`
Colour tab (UI)	2	`colourRefMode`, `colour_hold_a_b`
Polar (UI)	1	`polar_show_l/c/r/sum`

11 Known issues and roadmap

11.1 Known issues and surprises

Table 3. Known issues found while writing this document (K) and from the engine findings (F).

#	Issue	Evidence
K1	The ORTF pair is crossed: L sits at lower X, but −55° aims it toward +X. A source at x = 1 m gives cardioid gain L 0.539 vs R 0.963, so the stereo image may be mirrored against the plan. The comment in `angles_from_toe_half_deg` claims the "same sign convention" as the Reaktor parity default, which is actually the opposite.	VERIFIED numbers; ASSUMED not listened to
K2	The air 1-pole corner fc = Wet/d is about 133 Hz at 3 m. At a mix of 0.25 this is a broadband −2.5 dB step. The units may have been misread from the Reaktor graph.	VERIFIED maths; ASSUMED cause
K3	The default room gives only 12 distinct FDN delays out of 32 (source and mic centred in X).	VERIFIED
K4	`early_wet` = 0 and `fader_early` = 0 by default: the early field is off out of the box.	VERIFIED
K5	Server and client defaults disagree: space 0.25/0.17, humidity 0/0.35, decay_lock false/true, decay_s 0/1.8, mirror_b, and yaw None/∓55. Five routes, including `/api/defaults`, are never called by the UI.	VERIFIED
K6	Positive Dffs is unstable, and `diffusion_master` is not clamped. It is −0.25 in `fdn.py` but −0.5 live.	VERIFIED
K7	The card `diffusion` key drives both the Surfaces jitter and the Structure Dffs (P0.3).	VERIFIED
K8	The roadmap (P0.5d) and the C plan say Render uses `scipy.signal.lfilter`, but the software has no scipy import. The tests assert that scipy is absent, and the biquads are pure numpy.	VERIFIED contradiction
K9	`fdn_stub.py` says "delays from room size, not image delays", but the code sets delays from image distances.	VERIFIED
K10	Render early has no C capsule, while Play's C bus early is uncoloured. The two paths differ.	VERIFIED
K11	Play colour switches on only if the colour bank sample rate equals the AudioContext rate. Otherwise it is silently off.	VERIFIED
K12	`loop_mix` exists but is not packed or used.	VERIFIED
F1	No denormal guard: silence costs 4.03 ms/block in WASM (over budget) vs 0.56 ms with a guard.	VERIFIED, engine findings
F2	Proposed: per-line absorption (Jot), lg_i = 10^(−3·d_i/(sr·T60)), with g = 1/√N.	VERIFIED, engine findings
F3	Play delays are rounded at the server rate, not the Play rate. This causes a −3.35 dB direct error and an HF T30 loss.	VERIFIED, engine findings
F4	Play coefficients are designed at the server rate. Corners are ×1.088 off at 48 kHz.	VERIFIED, engine findings
F5	A same-sample loop solve in Play costs +0.16–0.27 ms. With F1–F3, the Play/Render null reaches −47.5 dB.	VERIFIED, engine findings
Tail	The tail is 1.5–6× longer than the decay dial, with a 22–25 s residue at 9/11 kHz. Tests t01–t18 are in `build_support/engine/tests`.	VERIFIED, engine findings

11.2 Roadmap

`BUILD_ROADMAP.md` orders the work as follows PLANNED:

- **P0:** engine fixes.
- **P1:** the Blob v2 bump.
- **P2–P6:** air, materials, panel Solo/Mute, the Play per-hit cascade, scattering and source radiation, in any order.
- **P7:** C kernels and a variable N.

- **P8:** the plugin.

With P4, P5 and P6 all on, two sources would use about 90 % of the CPU budget. That is why cheaper mic colour matters, and it is the work Build is doing now.

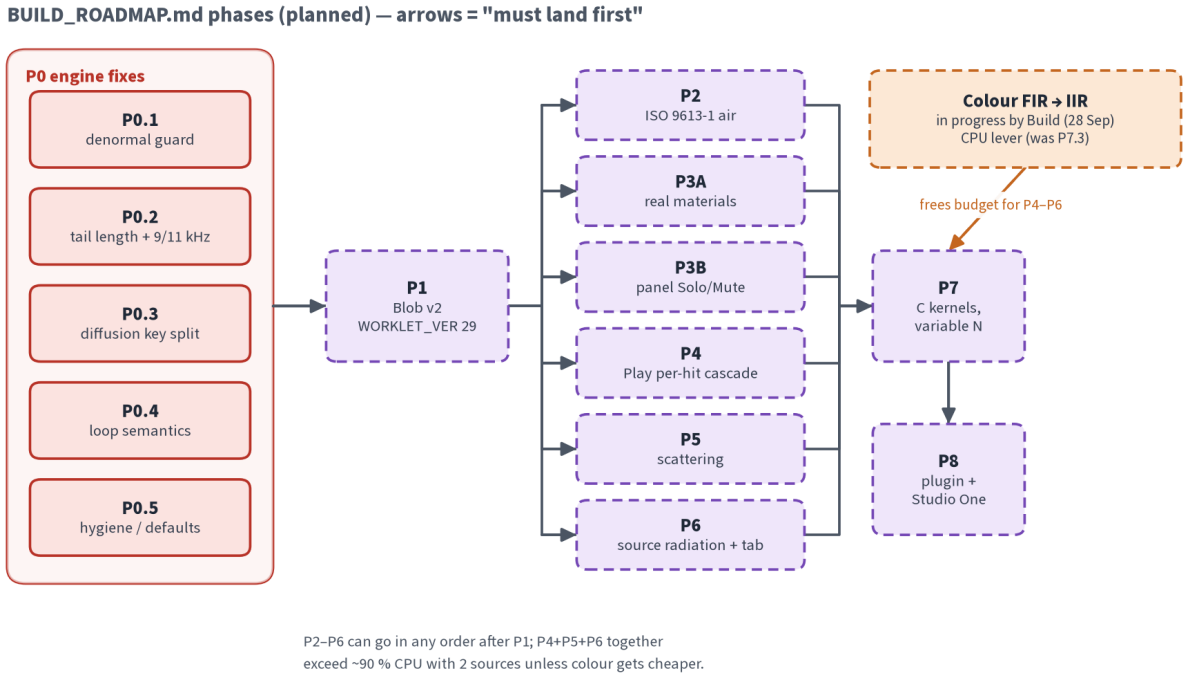


Figure 24. Roadmap dependency graph (planned).

For P8, the C plan recommends Design A PLANNED, ASSUMED design:

- A Source insert on each track renders its own direct and early sound. It sends its FDN inject through a normal host send to one Room instance on an FX channel.
- The Room instance owns the geometry, surfaces, the single FDN tail and the mic colour.
- The web app edits the shared room through an in-process registry.
- Studio One 7 added CLAP. Studio One 8 is now sold as Fender Studio Pro.

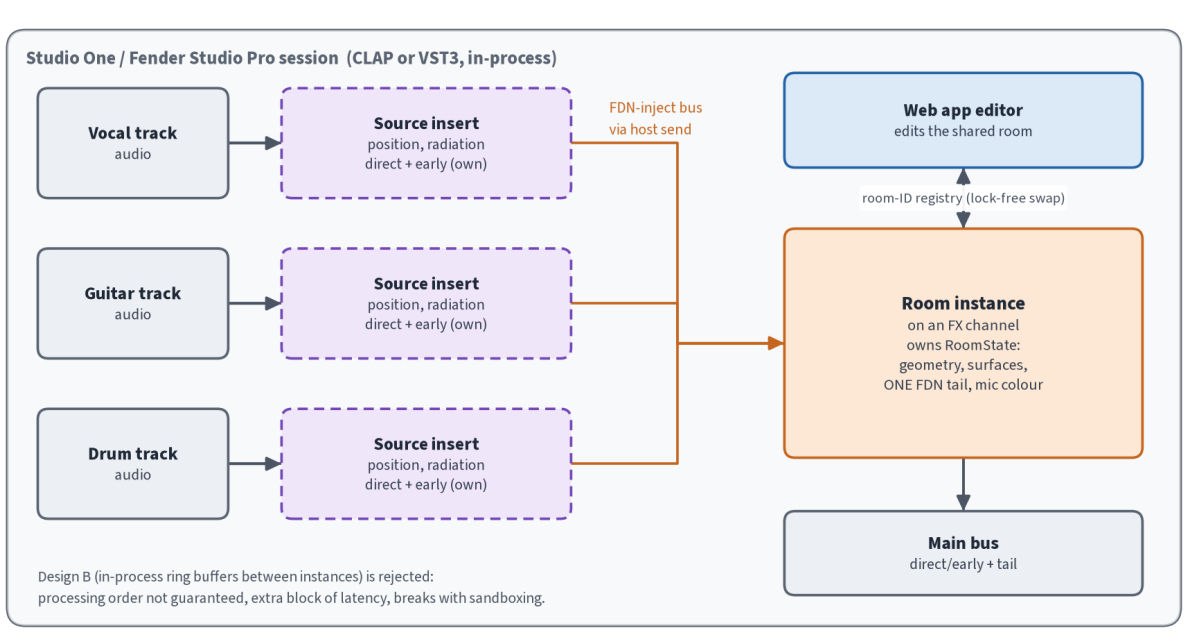


Figure 25. Planned Studio One plugin layout (Design A).

Appendix Sources

- Code: snapshot of `Desktop\2011 Room Reverb\software` taken 28 Sep 2026, about 09:05 BST (byte-identical to the 27 Sep 2026 snapshot).
- `mic_fitter/` (presets, README, reports) and its plots (the SM57 colour figure).
- `build_support/planning/`: BUILD_ROADMAP.md, C_MIGRATION_PLAN.md, PARAMETERS.md, parameters.csv.
- `build_support/research/` (cheap_colour and others) and `build_support/engine/` (findings F1–F5, tests t01–t18).
- Diagram sources: `diagrams/src/*.py` (matplotlib), next to the PNGs.